K.R. MANGALAM UNIVERSITY

Database Management Lab File
(ENCS205)

Bachelor of Technology

in

Computer Science and Engineering

Submitted by

Gaurav (2401010009)

B. Tech CSE (Core) – 4th Semester

Section: A

Submitted To:

MS. Mansi Kajal

School of Engineering & Technology
K. R. MANGALAM UNIVERSITY
Sohna, Haryana 122103, India Jan-2026

Index

S.NO	EXPERIMENT	PAGE NO.	DATE	REMARKS
1.	To analyze the requirements for a College Database, design an EntityRelationship (ER) Diagram representing Students, Sections, Subjects, and Marks, and convert the conceptual model into a Relational Database Schema.			
2.	To analyze the requirements for a Company Database, design an Entity-Relationship (ER) Diagram representing Employees, Departments, Projects, and their relationships (Works On, Manages, Supervision), and map the ER diagram into a normalized Relational Database Schema.			
3.	Database Schema for a customer-sale scenario.			

Experiment: 1

1.Consider following databases and draw ER diagram and convert entitiesand relationships to relation table for a given scenario:

Aim: To analyze the requirements for a College Database, design an Entity-Relationship (ER) Diagram representing Students, Sections, Subjects, and Marks, and convert the conceptual model into a Relational Database Schema.

COLLEGE DATABASE:

STUDENT (USN, SName, Address, Phone, Gender)

SEMSEC (SSID, Sem, Sec)

CLASS (USN, SSID)

SUBJECT (Subcode, Title, Sem, Credits)

IAMARKS (USN, Subcode, SSID, Test1, Test2, Test3, FinalIA)

PROCEDURE:

Step 1: Create the Strong Entities (Rectangles) Draw four rectangles to represent the main tables in your database:

1. **STUDENT**
2. **SEMSEC** (Semester & Section)
3. **SUBJECT**
4. **IAMARKS** (Internal Assessment Marks)

Step 2: Add Attributes (Ovals) Draw ovals connected to their respective entities. Make sure to identify Primary Keys and special attributes.

- For STUDENT:
  - Draw ovals: SNAME, ADDRESS, PHONE, GENDER.
  - Draw oval USN and **underline** the text (Primary Key).
- For SEMSEC:
  - Draw ovals: SEM, SEC.
  - Draw oval SSID and **underline** the text (Primary Key).
- For SUBJECT:
  - Draw ovals: Title, SEM, CREDITS.
  - Draw oval SUBCODE and **underline** the text (Primary Key).

- For IAMARKS:
    - Draw ovals: Test1, Test2, Test3.
    - Draw a Dashed/Dotted Oval for FinalIA. (This indicates it is a Derived Attribute, calculated from the other test scores).

Step 3: Create Relationships (Diamonds) Draw four diamond shapes to connect the entities.

1. Draw "CLASS" Diamond:
    - Place it between STUDENT and SEMSEC.
    - Meaning: Represents which class/section a student belongs to.
2. Draw "Secures" Diamond:
    - Place it between STUDENT and IAMARKS.
    - Meaning: Represents the marks secured by a specific student.
3. Draw "CLASS_MARKS" Diamond:
    - Place it between SEMSEC and IAMARKS.
    - Meaning: Links the marks to a specific section (to identify which class the marks belong to).
4. Draw "SCORED_IN" Diamond:
    - Place it between SUBJECT and IAMARKS.
    - Meaning: Identifies which subject the marks are for.

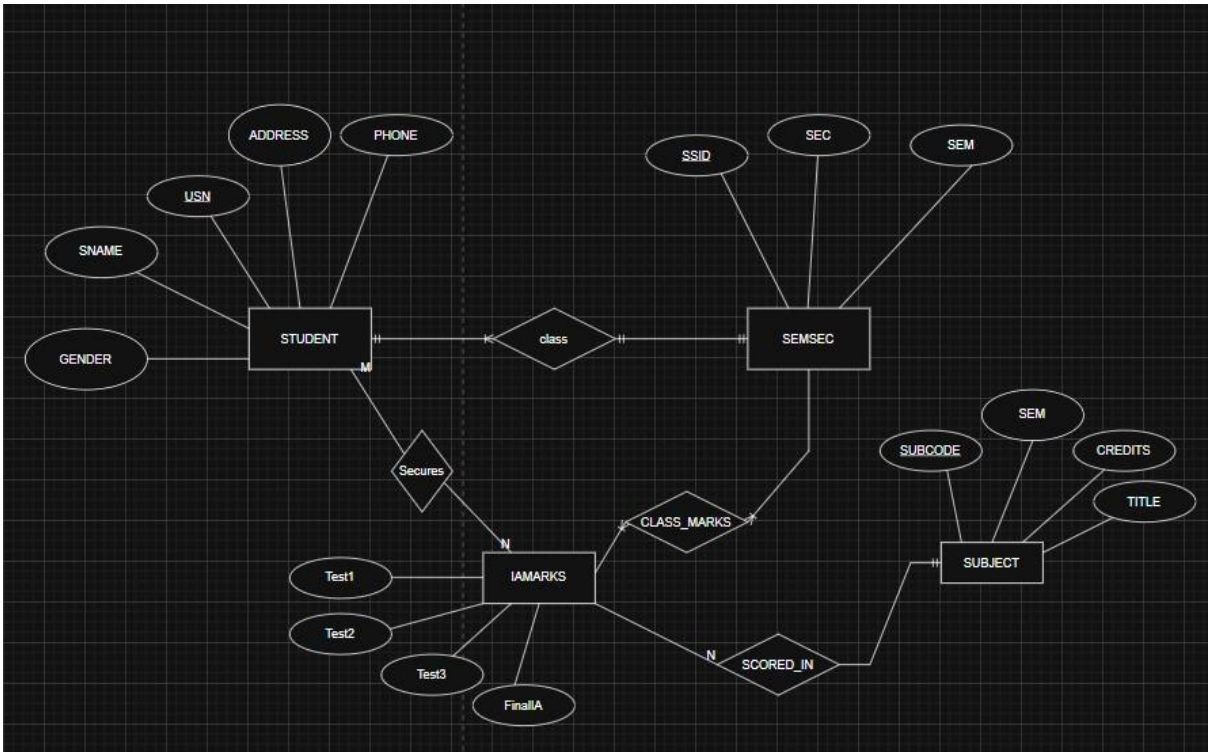
Step 4: Connect Lines & Define Cardinality (Mapping) Draw lines connecting the diamonds to the rectangles and label the cardinality (1, M, N) as shown in your image.

- For CLASS (Student - SemSec):
    - Draw a single line from STUDENT (Label 1). ○ Draw a double line (or crow's foot) from SEMSEC (Label N or (1, n)). ○ *Logic:* One Student is in One Section; One Section has Many Students.
- For Secures (Student - IAMarks): ○ Connect STUDENT to IAMARKS. ○ Label as M: N (Many-to-Many).
- For SCORED_IN (Subject - IAMarks):
    - Connect SUBJECT to IAMARKS.
    - Label as 1: N (One Subject has Many Marks entries).

- For CLASS_MARKS (SemSec - IAMarks):

- Connect SEMSEC to IAMARKS.
- Label as 1: N (One Section has Many Marks entries).

Step 5: Final Review Check that FinalIA is dashed (derived), all Primary Keys (USN, SSID, SUBCODE) are underlined, and all 4 entities are connected through the central IAMARKS relationships



EXPERIMENT: 2

2. **Consider following databases and draw ER diagram and convert entities and relationships to relation table for a given scenario:**

AIM: To analyze the requirements for a Company Database, design an Entity-Relationship (ER) Diagram representing Employees, Departments, Projects, and their relationships (Works On, Manages, Supervision), and map the ER diagram into a normalized Relational Database Schema.

PROCEDURE:

Step 1: Draw the 3 Main Entities (Rectangles)

Draw three rectangular boxes in the following positions:

1. DEPARTMENT: Place it in the top-middle.
2. Employee: Place it on the right side.

3. PROJECT: Place it in the bottom-left.

Step 2: Add Attributes (Ovals)

Draw ovals connected to their respective rectangles. Crucially, include the Foreign Keys as shown in your image:

- For DEPARTMENT:
  - Add ovals: DNAME, MGRSSN, MGRSTARTDATE.
  - Add oval DNO and underline it (Primary Key).
- For Employee:
  - Add ovals: NAME, ADDRESS, SALARY, SEX. ○ Add oval SSN and underline it (Primary Key).
  - Add Foreign Keys: Add ovals for SUPERSSN and DNO attached to this entity.
- For PROJECT:
  - Add ovals: PNAME, PLOCATION.
  - Add oval PNO and underline it (Primary Key).
  - Add Foreign Key: Add an oval for DNO attached to this entity.
- For Locations:
  - Draw a separate oval labeled LOOC below the Department table.

Step 3: Draw Relationships (Diamonds)

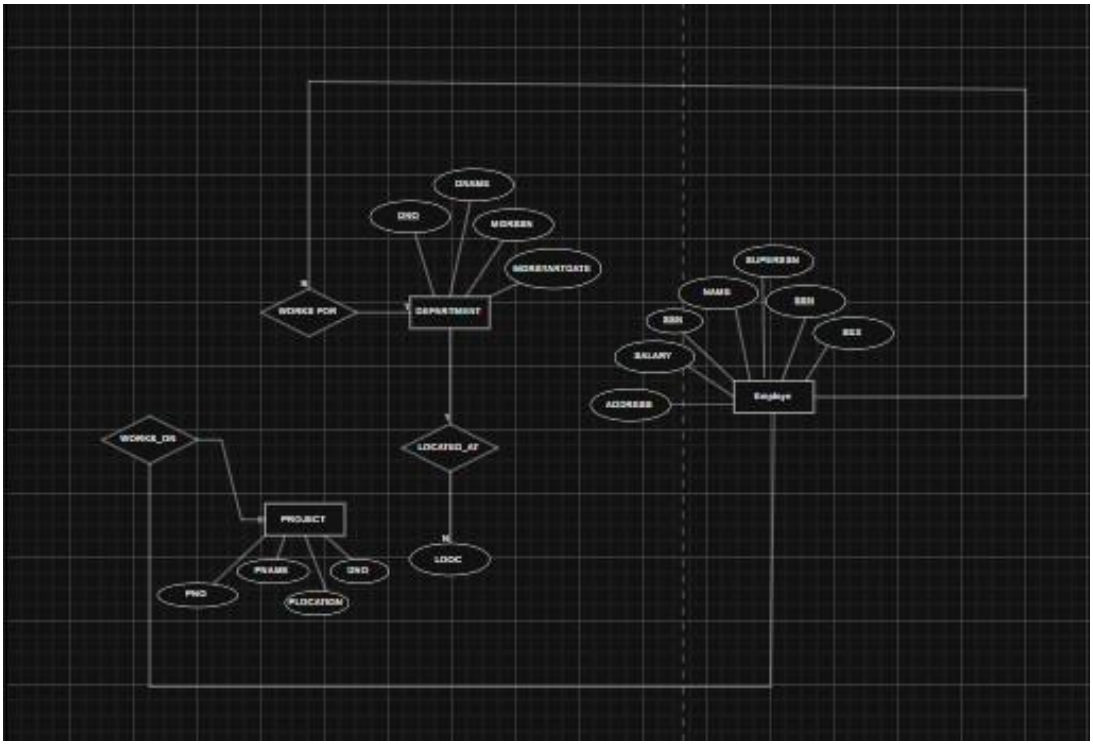
Draw three diamond shapes to represent the connections:

1. WORKS_FOR: Place this between DEPARTMENT and Employee.
2. LOCATED_AT: Place this between DEPARTMENT and the LOOC oval.
3. WORKS_ON: Place this to the left of the PROJECT table.

Step 4: Connect Lines & Define Cardinality

1. Connect WORKS_FOR:
   - Draw a line from DEPARTMENT to the diamond. Label it 1 (or N depending on your specific image logic, usually Many Employees work for 1 Dept). *In your image, N is near Employee and 1 is near Dept.*
   - Draw a line from Employee to the diamond**.**
2. Connect LOCATED_AT:
   - Draw a line from DEPARTMENT down to the diamond. Label it 1. ○ Draw a line from the diamond down to the LOOC oval. Label it N.
3. Connect WORKS_ON:

- Draw a line from PROJECT to the diamond.
- Draw a long line from the other side of the diamond that goes Up, turns Right (across the top of the page), turns Down, and connects to the Employee table.
- This represents the Many-to-Many relationship between Employees and Projects.



EXPERIMENT:3

3.Database Schema for a customer-sale scenario

Customer (Cust id: integer, cust_name: string)

Item (item_id: integer, item_name: string, price: integer)

Sale (bill_no:integer,bill_data:date,cust_id: integer, item_id: integer, qty_sold: integer)

For the above schema, perform the following—

1. Create the tables with the appropriate integrity constraints.

2. Insert around 10 records in each of the tables.

3. List all the bills for the current date with the customer names and item numbers.

4. List the total Bill details with the quantity sold, price of the item and the final amount.

5. List the details of the customer who have bought a product which has a price>200.

6. Give a count of how many products have been bought by each customer

7. Give a list of products bought by a customer having cust_id as 5.

8. List the item details which are sold as of today.

9. Create a view which lists out the bill_no, bill_date, cust_id, item_id, price, qty_sold, amount.

10. Create a view which lists the daily sales date wise for the last one week

```
CREATE DATABASE IF NOT EXISTS sales_db;
USE sales_db;

DROP VIEW IF EXISTS Weekly_Sales;
DROP VIEW IF EXISTS Bill_Details;

DROP TABLE IF EXISTS Sale;
DROP TABLE IF EXISTS Item;
DROP TABLE IF EXISTS Customer;

CREATE TABLE Customer (
   cust_id INT PRIMARY KEY,
   cust_name VARCHAR(50) NOT NULL
   );

CREATE TABLE Item (
   item_id INT PRIMARY KEY,
   item_name VARCHAR(50) NOT NULL,
   price INT NOT NULL CHECK (price > 0)
);

CREATE TABLE Sale (
   bill_no INT,
   bill_date DATE NOT NULL,
   cust_id INT,
   item_id INT,
   qty_sold INT NOT NULL CHECK (qty_sold > 0),
   PRIMARY KEY (bill_no, item_id),
   FOREIGN KEY (cust_id) REFERENCES Customer(cust_id),
```

```
    FOREIGN KEY (item_id) REFERENCES Item(item_id)
);
INSERT INTO Customer VALUES
(1,'Gaurav'),
(2,'Rahul'),
(3,'Sneha'),
(4,'Priya'),
(5,'Karan'),
(6,'Neha'),
(7,'Vikas'),
(8,'Rohit'),
(9,'Pooja'),
(10,'Anjali');

INSERT INTO Item VALUES
(101,'Pen',50),
(102,'Notebook',120),
(103,'Bag',500),
(104,'Bottle',250),
(105,'Shoes',1500),
(106,'Pencil',20),
(107,'Scale',30),
(108,'Watch',2000),
(109,'T-shirt',800),
(110,'Headphones',1800);

INSERT INTO Sale VALUES
(1000,CURRENT_DATE,1,101,2),
(1001,CURRENT_DATE,1,103,1),
```

```
(1002,CURRENT_DATE,2,105,1),
(1003,CURRENT_DATE,3,104,3),
(1004,DATE_SUB(CURRENT_DATE, INTERVAL 1 DAY),5,108,1),
(1005,DATE_SUB(CURRENT_DATE, INTERVAL 2 DAY),5,102,4),
(1006,CURRENT_DATE,6,109,2),
(1007,CURRENT_DATE,7,110,1),
(1008,DATE_SUB(CURRENT_DATE, INTERVAL 3 DAY),8,103,1),
(1009,CURRENT_DATE,9,104,2);

SELECT s.bill_no, c.cust_name, s.item_id
FROM Sale s
JOIN Customer c ON s.cust_id = c.cust_id
WHERE s.bill_date = CURRENT_DATE;

SELECT s.bill_no,
    s.item_id,
    s.qty_sold,
    i.price,
    (s.qty_sold * i.price) AS Final_Amount
FROM Sale s
JOIN Item i ON s.item_id = i.item_id;

SELECT DISTINCT c.*
FROM Customer c
JOIN Sale s ON c.cust_id = s.cust_id
JOIN Item i ON s.item_id = i.item_id
WHERE i.price > 200;

SELECT c.cust_name,
```

```
    SUM(s.qty_sold) AS Total_Products
FROM Customer c
JOIN Sale s ON c.cust_id = s.cust_id
GROUP BY c.cust_name;

SELECT i.item_name
FROM Sale s
JOIN Item i ON s.item_id = i.item_id
WHERE s.cust_id = 5;

SELECT DISTINCT i.*
FROM Item i
JOIN Sale s ON i.item_id = s.item_id
WHERE s.bill_date = CURRENT_DATE;

CREATE VIEW Bill_Details AS
SELECT s.bill_no,
    s.bill_date,
    s.cust_id,
    s.item_id,
    i.price,
    s.qty_sold,
    (s.qty_sold * i.price) AS amount
FROM Sale s
JOIN Item i ON s.item_id = i.item_id;

CREATE VIEW Weekly_Sales AS
SELECT s.bill_date,
    SUM(s.qty_sold * i.price) AS Total_Sales
```

```
FROM Sale s
JOIN Item i ON s.item_id = i.item_id
WHERE s.bill_date >= DATE_SUB(CURRENT_DATE, INTERVAL 7 DAY)
GROUP BY s.bill_date;

SELECT * FROM Bill_Details;
SELECT * FROM Weekly_Sales;
```

OUTPUT:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

bill_no	cust_name	item_id
1000	Gaurav	101
1001	Gaurav	103
1002	Rahul	105
1003	Sneha	104
1006	Neha	109
1007	Vikas	110
1009	Pooja	104

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

bill_no	item_id	qty_sold	price	Final_Amount
1000	101	2	50	100
1001	103	1	500	500
1002	105	1	1500	1500
1003	104	3	250	750
1004	108	1	2000	2000
1005	102	4	120	480
1006	109	2	800	1600
1007	110	1	1800	1800
1008	103	1	500	500
1009	104	2	250	500

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

cust_id	cust_name
1	Gaurav
8	Rohit
3	Sneha
9	Pooja
2	Rahul
5	Karan
6	Neha
7	Vikas

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

cust_name	Total_Products
Gaurav	3
Rahul	1
Sneha	3
Karan	5
Neha	2
Vikas	1
Rohit	1
Pooja	2

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

item_name
Watch
Notebook

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

item_id	item_name	price
101	Pen	50
103	Bag	500
105	Shoes	1500
104	Bottle	250
109	T-shirt	800
110	Headphones	1800

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

bill_no	bill_date	cust_id	item_id	price	qty_sold	amount
1000	2026-03-01	1	101	50	2	100
1001	2026-03-01	1	103	500	1	500
1002	2026-03-01	2	105	1500	1	1500
1003	2026-03-01	3	104	250	3	750
1004	2026-02-28	5	108	2000	1	2000
1005	2026-02-27	5	102	120	4	480
1006	2026-03-01	6	109	800	2	1600
1007	2026-03-01	7	110	1800	1	1800
1008	2026-02-26	8	103	500	1	500
1009	2026-03-01	9	104	250	2	500

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

bill_date	Total_Sales
2026-03-01	6750
2026-02-28	2000
2026-02-27	480
2026-02-26	500